

Participant Handbook

Clinical Data Analysis in R - Phase I: Introduction to R for Clinical Research

Trainers: Bernard Osang'ir (Senior Biostatistician, lead), My Luong Vuong & Bui Cuong

Schedule: Five evening sessions · every Tuesday, 8:00 PM, 90 minutes · 8 September – 6 October 2026

Organisation: Neudata

Hashtag: #ClearDataClearImpact

Welcome

Welcome to your five-session journey into clinical data analysis with R. This course is built for clinicians, nurses, pharmacists, public-health practitioners and health researchers who have never written a line of code. You do not need a mathematics or computing background. You need curiosity, your clinical reasoning, and a willingness to make mistakes and learn from them. Everyone in the room started exactly where you are.

By the end of the week you will take a messy, real-world clinical dataset, clean it, describe it, test hypotheses, build a logistic regression model, and write up the results the way a journal expects. That is a complete analysis, start to finish, done reproducibly so that anyone (including future you) can re-run it and get the same answer.

This handbook is your self-study companion. It mirrors the live demonstrations you will see each day, explains the concepts in plain language, and shows you what the R output means. Keep it open while you follow along in class, and return to it afterwards to revise.

How to use this handbook

- **During class:** follow the worked code blocks alongside the instructor. Type the code yourself rather than copying and pasting. The muscle memory matters.
- **After class:** re-read the chapter for the day, run the code again on your own machine, and attempt the exercise at the end of the chapter.
- **For reference:** use the glossary, the "Common errors" section, and the separate `R_command_reference_sheet.md` cheat sheet whenever you get stuck.
- **Code blocks** look like this and are meant to be run in R:

```
mean(c(120, 130, 145, 150)) # the average of four systolic readings
```

- **Comments** begin with a #. R ignores everything after the # on a line; comments are notes for humans.
- A short **"What the output means"** note follows the important code so you can interpret what you see in the Console.

A practical tip used throughout the course: define styling and settings once and reuse them. For example, our figures all use a single teal brand colour:

```
course_teal <- "#0D7377" # define once, reuse in every plot
```

What you will be able to do (the 13 objectives)

By the end of Phase I you will be able to:

1. Navigate R and RStudio confidently, using the Console, the Script editor, and Projects.
2. Create and use objects, vectors, and data frames.
3. Install and load packages, especially the tidyverse.
4. Import clinical data from CSV and Excel files.
5. Inspect a dataset and recognise data-quality problems on sight.
6. Identify variable types and convert between them (numeric, character, factor, date).
7. Clean data: handle missing values, remove duplicates, fix inconsistent categories, and validate impossible values.
8. Derive new variables (for example BMI and clinical categories) and set factor reference levels.
9. Compute descriptive statistics correctly, including the proper handling of missing data.
10. Build a manuscript-ready "Table 1" and publication-quality figures, and export them.
11. Choose and run the right hypothesis test for a given question, and interpret p-values honestly.
12. Fit univariable and multivariable logistic regression models and interpret odds ratios with 95% confidence intervals.
13. Build, diagnose, and report a final model reproducibly, and write a Results section in clinical-manuscript style.

Getting Set Up

Installing R and RStudio

R is the engine; RStudio is the comfortable dashboard you drive it from. You install both, and you install R first.

1. **Install R** from the Comprehensive R Archive Network (CRAN) at <https://cran.r-project.org>. Choose the version for your operating system (Windows, macOS, or Linux).
2. **Install RStudio Desktop** (the free edition) from <https://posit.co/download/rstudio-desktop>.
3. Open RStudio. You should never need to open the plain R program directly; RStudio runs R for you.

For step-by-step screenshots and troubleshooting, see the separate `package_installation_guide.md` provided with the course materials.

The four RStudio panes

When RStudio opens you see (up to) four panes. Knowing what each one does removes most early confusion.

Pane	Location (default)	What it is for
Source / Script editor	Top-left	Where you write and save your code. This is what you keep and re-run.
Console	Bottom-left	Where code runs and results appear. You can type here too, but it is not saved.
Environment / History	Top-right	Lists the objects (data, variables) currently in memory.
Files / Plots / Packages / Help	Bottom-right	Browse files, view figures, manage packages, read help pages.

The single most important habit: **write your code in the Script editor, not the Console**. The script is your permanent, reproducible record. To run the current line, press `Ctrl+Enter` (Windows) or `Cmd+Enter` (Mac). The result appears in the Console.

Projects and working directories

The **working directory** is the folder R is "looking at" right now. If your code says `read_csv("Data/file.csv")`, R looks for a Data folder inside the working directory.

The professional way to manage this is an **RStudio Project**. Go to `File > New Project` and point it at your course folder (the one containing `Data/`, `Scripts/`, and so on). From then on, every path is relative to that project root, and your analysis will run on any computer.

```
getwd() # shows the current working directory
```

Avoid `setwd("C:/Users/yourname/somewhere")`. A hard-coded path like that breaks the moment anyone else (or a different machine) runs your code. Relative paths inside a Project are portable; absolute paths are not.

Installing and loading packages

A **package** is a bundle of extra functions someone wrote and shared. You **install** a package once (it downloads from the internet), and you **load** it with `library()` in every new session.

```
# install.packages("tidyverse") # run ONCE per machine (already done in class)
library(tidyverse)             # run EVERY session to make the tools available
```

The single biggest beginner mistake here is running `install.packages()` every time. That is slow and needs internet. Install once; `library()` each session. For the full list of packages this course uses and how to install them, see `package_installation_guide.md`.

The Case Study and the Dataset

Everything in this course revolves around one realistic (simulated) study, so you learn the whole pipeline on a single coherent problem.

The study

Title: Determinants of Hypertension Treatment Uptake among Adults attending Primary Healthcare Facilities.

Design: A multicentre cross-sectional study of 1,500 adults attending six primary healthcare facilities: Bugando PHC, Kisesa HC, Nyamagana PHC, Ilemela HC, Buzuruga PHC, and Igoma HC.

Research question: Among adults already diagnosed with hypertension, what factors determine whether they are actually on antihypertensive treatment?

Primary outcome: `treatment_uptake` (currently on antihypertensive therapy: Yes/No). This is analysed only among the roughly 1,089 patients with diagnosed hypertension (`htn_diagnosed == "Yes"`). You cannot "take up" treatment for a disease you have not been diagnosed with.

The raw data file deliberately contains the kinds of problems you meet in real clinical data: it has **1,503 rows** (three duplicate records) instead of 1,500, mixed category spellings, mixed binary codings, missing-value sentinels, impossible physiological values, stray whitespace, and mixed date formats. Cleaning these is the heart of Day 2. The cleaned, analysis-ready data is saved as `Data/analysis_data.rds`, and every later day starts from that single clean file.

The dataset (36 variables, by group)

The full data dictionary is in `Data/data_dictionary.md`. Here is the map by group.

Demographic and social variables

Variable	Meaning
<code>patient_id</code>	Unique identifier (PHC-0001 ... PHC-1500)
<code>facility</code>	One of the six PHC facilities
<code>enroll_date</code>	Date of enrolment (mixed formats in raw file)
<code>age</code>	Age in years
<code>sex</code>	Female / Male
<code>residence</code>	Urban / Rural
<code>education</code>	Ordered: None < Primary < Secondary < Tertiary
<code>occupation, marital_status</code>	Social categoricals
<code>health_insurance</code>	Yes / No

Behavioural variables

Variable	Meaning
<code>smoking</code>	Never / Former / Current
<code>alcohol</code>	None / Moderate / Heavy
<code>physical_activity</code>	Ordered: Low / Moderate / High

Clinical variables

Variable	Meaning
height_cm, weight_kg, bmi	Anthropometry; BMI recomputed from height and weight
family_history_htn	Family history of hypertension (Yes/No)
diabetes	Diabetes mellitus (Yes/No)
sbp_mmhg, dbp_mmhg	Systolic and diastolic blood pressure
knowledge_score	Hypertension knowledge, 0-20
distance_to_facility_km	Distance to the facility
comorbidity_count	Number of comorbidities
htn_diagnosed	Diagnosed hypertensive (defines the analysis population)
months_since_diagnosis	Months since diagnosis

Laboratory biomarkers (all numeric, in their stated units)

total_chol_mmol_l, hdl_mmol_l, ldl_mmol_l, triglycerides_mmol_l,
fasting_glucose_mmol_l, creatinine_umol_l, sodium_mmol_l, potassium_mmol_l.

Outcomes

Variable	Meaning
treatment_uptake	PRIMARY outcome: on antihypertensive treatment (Yes/No)
adherence	Good / Poor (only meaningful among the treated)
bp_controlled	SBP<140 and DBP<90 among the treated

Keep the data dictionary beside you. Knowing what each variable means is half of good analysis.

Chapter 1 (Day 1): R Basics

Day 1 is about getting comfortable. We treat R as a calculator, learn to store values, build vectors and data frames, load packages, import the clinical data, and take a first look at it.

R as a calculator

Everything that runs appears in the Console. Type arithmetic and R answers.

```
2 + 2
140 / 90          # a blood-pressure ratio, just to show division
sqrt(16)
mean(c(120, 130, 145, 150)) # mean of four systolic readings
```

What the output means: R prints 4, 1.555..., 4, and 136.25. The last line shows two ideas at once: `c()` builds a collection of values, and `mean()` is a function that summarises them.

Objects: storing a value

You store a value in an **object** using the assignment arrow `<-`. Read it as "gets".

```
sbp <- 152      # systolic BP of one patient; sbp "gets" 152
sbp            # type the name to print it
age <- 60
sbp + 10       # objects behave like the values they hold -> 162
```

Two things to remember from the start:

- Use `<-` for assignment (the keyboard shortcut is `Alt+-`). Using `=` mostly works but `<-` is the R convention.
- R is **case-sensitive**: SBP is a different object from sbp. A surprising number of early errors are just a capital letter in the wrong place.

Vectors: many values of the same type

A **vector** holds several values, all of the same type. The `c()` function ("combine") builds one.

```
sbp_readings <- c(152, 138, 145, 160, 129, 142)
sbp_readings
length(sbp_readings) # how many values?
mean(sbp_readings)
sd(sbp_readings)
max(sbp_readings)
summary(sbp_readings) # min, quartiles, median, mean, max
```

Vectors can be text (character) or true/false (logical), not just numbers:

```
sex <- c("Female", "Male", "Female", "Female", "Male", "Female")
high_bp <- sbp_readings >= 140 # a TRUE/FALSE for each reading
high_bp
sum(high_bp) # TRUE counts as 1, so this counts the highs
```

What the output means: `high_bp` is TRUE FALSE TRUE TRUE FALSE TRUE, and `sum(high_bp)` is 4 because four readings were at least 140. Adding up a logical vector to count "how many" is a trick you will use constantly.

Packages

```
library(tidyverse) # data import, wrangling, and ggplot2 for figures
library(readxl)    # read Excel files
```

The tidyverse is the everyday toolkit for this course.

Data frames and importing clinical data

A **data frame** is a table: rows are patients, columns are variables. It is the central object in all clinical analysis. We import ours from disk.

```
# CSV (comma-separated values)
htn <- read_csv("Data/hypertension_phc_raw.csv")

# The same data as Excel, just to show readxl
htn_xl <- read_excel("Data/hypertension_phc_raw.xlsx", sheet = "data")
```

What the output means: `read_csv()` prints a column specification and Rows: 1503 Columns: 36. Note 1503, not 1500 - the three duplicate rows are already lurking. R is telling you to clean before you analyse.

First inspection

Look at the data before you do anything with it.

```
htn          # a tibble prints the first 10 rows neatly
dim(htn)      # rows, columns
nrow(htn); ncol(htn)
names(htn)    # variable names
head(htn, 5)  # first 5 rows
glimpse(htn)  # compact structure: the type of every column
# View(htn)   # opens the spreadsheet viewer (run inside RStudio)
```

Then look at single variables. The `$` pulls one column out as a vector.

```
htn$age
summary(htn$age)    # NOTE the max of 200 -> an impossible age
table(htn$sex)      # NOTE Female / F / female / f -> messy spellings
table(htn$facility) # the six facilities (some with stray spaces)
```

What the output means: even before any statistics, R is showing you that the data need cleaning - an impossible maximum age of 200, and sex recorded four different ways. That is exactly what Day 2 fixes. On Day 1 we save nothing; the goal is simply to get data in and look at it.

Key takeaways

- Write code in the Script editor; results appear in the Console.
- `<-` assigns; `c()` combines; R is case-sensitive.
- A data frame is a table of patients (rows) and variables (columns).
- `read_csv()` / `read_excel()` import data; `glimpse()`, `summary()`, and `table()` inspect it.
- Inspection often reveals data-quality problems straight away.

Day 1 exercise

Open `Practicals/day1_exercise.R`. Import the raw dataset, report its dimensions, list its variable names, and use `summary()` and `table()` on at least three variables to identify two data-quality problems you can see with your own eyes.

Chapter 2 (Day 2): Understanding and Cleaning Clinical Data

Day 2 turns the messy raw file into a tidy, analysis-ready dataset and **saves** it. This saved file is the "contract" for Days 3, 4, and 5: every later day starts from exactly the same clean data, so results are consistent and reproducible.

```
library(tidyverse)
```

Variable types

Every column has a **type**, and the type determines what you can do with it.

- **numeric** (also called double): numbers you can average, like age or sbp_mmhg.
- **character**: free text, like patient_id.
- **factor**: a categorical variable with a fixed set of **levels**, like sex (Female, Male). Factors are how R represents categories in tables and models.
- **logical**: TRUE / FALSE.
- **Date**: calendar dates R understands as time, not text.

Getting types right is not pedantry. A factor with the wrong reference level gives you a backwards odds ratio; a date stored as text cannot be sorted by time.

Importing with missing-value codes

Real datasets hide "missing" inside sentinel values like blanks, NA, 999, or -99. Tell `read_csv()` to treat all of them as missing from the start.

```
raw <- read_csv(
  "Data/hypertension_phc_raw.csv",
  na = c("", "NA", "999", "-99") # all of these become NA
)
glimpse(raw)
nrow(raw) # 1503 - duplicates still present
```

Removing duplicates

```
sum(duplicated(raw)) # how many fully duplicated rows?
raw <- distinct(raw) # drop exact duplicate rows
n_distinct(raw$patient_id) # should now equal nrow(raw)
nrow(raw) # 1500
```

What the output means: `sum(duplicated(raw))` is 3. After `distinct()`, the row count drops to 1,500 and the number of distinct IDs matches the number of rows - no patient is counted twice.

Inconsistent categories and whitespace

First strip stray spaces from every text column at once, then fix specific variables.

```
# Trim leading/trailing spaces from ALL character columns
raw <- raw |> mutate(across(where(is.character), str_trim))
```

The `|>` is the **pipe**: read it as "and then". It passes the result on the left into the function on the right, so you can chain steps in the order you think about them.

Now recode the messy sex variable into two clean levels using `case_when()`, which checks rules top to bottom:

```
raw <- raw |>
  mutate(sex = case_when(
    str_to_lower(sex) %in% c("female", "f") ~ "Female",
    str_to_lower(sex) %in% c("male", "m") ~ "Male",
    TRUE ~ NA_character_ # anything else -> missing
  ))
table(raw$sex, useNA = "ifany") # now just Female / Male
```

Several variables mix Yes/No, Y/N, and 1/0. Rather than repeat the logic, write a small reusable **helper function** and apply it to all of them with `across()`:

```
to_yesno <- function(x) {
  x <- str_to_lower(str_trim(as.character(x)))
  case_when(
    x %in% c("yes", "y", "1", "true") ~ "Yes",
    x %in% c("no", "n", "0", "false") ~ "No",
    TRUE ~ NA_character_
  )
}

raw <- raw |>
  mutate(across(c(diabetes, family_history_htn, health_insurance,
                  htn_diagnosed, treatment_uptake), to_yesno))
table(raw$treatment_uptake, useNA = "ifany")
```

Validating impossible values

Define plausible physiological ranges; anything outside becomes NA. The `if_else()` function keeps the value if the condition is true and substitutes `NA_real_` otherwise.

```
raw <- raw |>
  mutate(
    age = if_else(age >= 18 & age <= 110, age, NA_real_),
    sbp_mmhg = if_else(sbp_mmhg >= 70 & sbp_mmhg <= 260, sbp_mmhg, NA_real_),
    dbp_mmhg = if_else(dbp_mmhg >= 40 & dbp_mmhg <= 150, dbp_mmhg, NA_real_),
    height_cm = if_else(height_cm >= 120 & height_cm <= 210, height_cm, NA_real_),
    weight_kg = if_else(weight_kg >= 30 & weight_kg <= 200, weight_kg, NA_real_)
  )
summary(select(raw, age, sbp_mmhg, dbp_mmhg, height_cm, weight_kg))
```

What the output means: the impossible values flagged in the data dictionary (age 0 and 200, SBP 0 and 700, the 7 kg weight, the 17 cm height, the DBP of 5) are now NA, and the `summary()` maxima and minima fall inside clinically sensible bounds.

Recoding and deriving new variables

Recompute BMI from the cleaned height and weight (never trust the supplied `bmi`, which had errors), then create clinical categories with `cut()` and `case_when()`.

```
raw <- raw |>
mutate(
  bmi = round(weight_kg / (height_cm / 100)^2, 1), # recompute from source
  bmi_cat = cut(bmi,
                breaks = c(-Inf, 18.5, 25, 30, Inf),
                labels = c("Underweight", "Normal", "Overweight", "Obese")),
  bp_category = case_when(
    is.na(sbp_mmhg) | is.na(dbp_mmhg) ~ NA_character_,
    sbp_mmhg >= 140 | dbp_mmhg >= 90 ~ "Hypertension",
    sbp_mmhg >= 130 | dbp_mmhg >= 80 ~ "Elevated",
    TRUE ~ "Normal"
  )
)
```

Parsing mixed dates

The `enroll_date` column mixes formats. `lubridate::parse_date_time()` tries a list of formats in order.

```
library(lubridate)
raw <- raw |>
mutate(enroll_date = parse_date_time(
  enroll_date,
  orders = c("ymd", "dmy", "d-b-Y")) |> as_date())
sum(is.na(raw$enroll_date)) # how many failed to parse?
```

Factors and reference levels

Set the correct type for every categorical variable, and choose the **reference level** deliberately. For binary predictors the reference (the comparison group) is listed first; we put "No" first so models estimate the odds of "Yes".

```
analysis_data <- raw |>
  mutate(
    facility      = factor(facility),
    sex           = factor(sex, levels = c("Female", "Male")),
    residence      = factor(residence, levels = c("Rural", "Urban")),
    education      = factor(education,
                           levels = c("None", "Primary", "Secondary", "Tertiary"),
                           ordered = TRUE),
    occupation     = factor(occupation),
    marital_status = factor(marital_status),
    physical_activity = factor(physical_activity,
                              levels = c("Low", "Moderate", "High"), ordered = TRUE),
    smoking        = factor(smoking, levels = c("Never", "Former", "Current")),
    alcohol         = factor(alcohol, levels = c("None", "Moderate", "Heavy")),
    bp_category     = factor(bp_category,
                             levels = c("Normal", "Elevated", "Hypertension")),
    # Binary predictors/outcomes: reference level "No" comes FIRST
    health_insurance = factor(health_insurance, levels = c("No", "Yes")),
    family_history_htn = factor(family_history_htn, levels = c("No", "Yes")),
    diabetes         = factor(diabetes, levels = c("No", "Yes")),
    htn_diagnosed     = factor(htn_diagnosed, levels = c("No", "Yes")),
    treatment_uptake = factor(treatment_uptake, levels = c("No", "Yes")),
    adherence         = factor(na_if(adherence, ""), levels = c("Poor", "Good")),
    bp_controlled     = factor(na_if(bp_controlled, ""), levels = c("No", "Yes"))
  )
  glimpse(analysis_data)
```

Ordered factors (education, physical_activity) preserve their natural ranking, which lets a model estimate a trend across levels.

A final missing-data audit, then save

```
colSums(is.na(analysis_data)) |> sort(decreasing = TRUE) |> head(12)
```

What the output means: this lists the columns with the most missing values. adherence and bp_controlled are missing by design (they only apply to treated patients), so high missingness there is expected, not an error.

Finally, save the clean data. The .rds file preserves factor types exactly; the .csv is a human-readable backup.

```
saveRDS(analysis_data, "Data/analysis_data.rds") # preserves types
write_csv(analysis_data, "Data/analysis_data.csv") # readable backup

# To reload on later days:
# analysis_data <- readRDS("Data/analysis_data.rds")
```

Key takeaways

- Declare missing-value codes at import with `na = c(...)`.
- `distinct()` removes duplicate rows; check IDs are unique afterwards.
- `str_trim()`, `case_when()`, and a `to_yesno()` helper standardise messy categories.
- Validate impossible values into NA using `if_else()` and plausible ranges.
- Recompute derived variables (BMI) from clean inputs; never trust a supplied derived column.
- Set factor levels and reference categories on purpose; use ordered factors for natural rankings.

- Save the clean data once and start every later analysis from it.

Day 2 exercise

Open Practicals/day2_exercise.R. Starting from the raw file, reproduce the cleaning pipeline: import with missing codes, remove duplicates, clean sex and the binary variables, validate at least three numeric ranges, derive BMI and bmi_cat, set factors, audit missingness, and save your own analysis_data.rds.

Chapter 3 (Day 3): Descriptive Statistics, Tables, and Figures

Day 3 describes the sample - numerically and visually - and produces a manuscript "Table 1" and journal-quality figures. We always start from the saved clean data.

```
library(tidyverse)
library(gtsummary) # the gold standard for "Table 1" in R

course_teal <- "#0D7377"
if (!dir.exists("Resources")) dir.create("Resources")

analysis_data <- readRDS("Data/analysis_data.rds")
nrow(analysis_data) # 1500 patients
```

Central tendency and spread (and the na.rm trap)

Central tendency answers "where is the middle?" (mean, median). Spread answers "how scattered are the values?" (SD, IQR, range).

The single most common beginner trap: if a column has *any* missing value, `mean()` and `sd()` return NA. You must add `na.rm = TRUE` to tell R to ignore the missing values.

```
mean(analysis_data$sbp_mmhg) # may be NA if any value is missing
mean(analysis_data$sbp_mmhg, na.rm = TRUE) # the CORRECT way

mean(analysis_data$age, na.rm = TRUE)
median(analysis_data$age, na.rm = TRUE) # robust to outliers
sd(analysis_data$age, na.rm = TRUE)
IQR(analysis_data$age, na.rm = TRUE) # interquartile range (Q3 - Q1)
quantile(analysis_data$age,
  probs = c(0, 0.25, 0.5, 0.75, 1), na.rm = TRUE)
```

Clinical interpretation: when the mean and median are close, the distribution is roughly symmetric and the mean is a fair summary. When the mean sits well above the median (common for BP and BMI), the data are right-skewed - a few high values drag the mean up - so report the **median (IQR)** instead.

A grouped numeric summary

`across()` applies the same set of functions to many columns at once, with no copy-paste. Naming each statistic makes the output self-documenting.

```
numeric_summary <- analysis_data |>
  group_by(treatment_uptake) |>
  summarise(
    n = n(),
    across(
      c(age, bmi, sbp_mmhg, dbp_mmhg),
      list(
        mean = ~ mean(.x, na.rm = TRUE),
        sd   = ~ sd(.x,   na.rm = TRUE),
        median = ~ median(.x, na.rm = TRUE)
      ),
      .names = "{.col}_{.fn}"
    ),
    .groups = "drop"
  )
print(numeric_summary)
```

Clinical interpretation: compare the rows. If treated patients have a higher mean age and SBP, that hints older or sicker patients are the ones being started on treatment - a signal we test formally later.

Frequency tables and cross-tabs

```
table(analysis_data$sex)                # raw counts
prop.table(table(analysis_data$sex))    # proportions (sum to 1)
round(100 * prop.table(table(analysis_data$sex)), 1) # as percentages

# table() SILENTLY drops NA by default; make missing visible:
table(analysis_data$education, useNA = "ifany")

# tidyverse equivalent returns a tidy data frame you can pipe onward:
analysis_data |>
  count(education) |>
  mutate(percent = round(100 * n / sum(n), 1))
```

A **cross-tabulation** counts two categorical variables together. The `margin` argument of `prop.table()` decides which way percentages run.

```
xtab <- table(analysis_data$treatment_uptake, analysis_data$diabetes)
xtab
round(100 * prop.table(xtab, margin = 1), 1) # ROW %: each row sums to 100
round(100 * prop.table(xtab, margin = 2), 1) # COLUMN %: each column sums to 100
```

Clinical interpretation: choose the percentage that answers your question. "What proportion of diabetics are on treatment?" is a **column** percentage (`margin = 2`). "What proportion of treated patients are diabetic?" is a **row** percentage (`margin = 1`). Reporting the wrong margin is one of the most common errors in manuscripts.

Building a manuscript Table 1 with gtsummary

`gtsummary` turns raw data into the baseline-characteristics table every clinical paper opens with. It picks sensible summaries automatically (mean (SD) or median (IQR) for numerics, n (%) for categoricals) and labels missing data for you.

```

table1 <- analysis_data |>
  select(
    age, sex, residence, education, bmi, bmi_cat,
    smoking, alcohol, physical_activity,
    family_history_htn, diabetes,
    sbp_mmhg, dbp_mmhg, bp_category,
    total_chol_mmol_l, fasting_glucose_mmol_l,
    treatment_uptake
  ) |>
  tbl_summary(
    by = treatment_uptake,          # one column per outcome group
    missing_text = "(Missing)",
    label = list(
      age ~ "Age (years)",
      sex ~ "Sex",
      bmi ~ "BMI (kg/m^2)",
      sbp_mmhg ~ "Systolic BP (mmHg)",
      dbp_mmhg ~ "Diastolic BP (mmHg)"
    )
  ) |>
  add_p() |>          # add a p-value column comparing the groups
  add_overall() |>    # add a total column
  bold_labels() |>
  modify_caption("***Table 1. Baseline characteristics by treatment uptake**")

table1 # prints in the Viewer

```

To export it (for a manuscript):

```

# To Word:
# table1 |> as_flex_table() |>
#   flextable::save_as_docx(path = "Resources/table1_demo.docx")
# To HTML:
# table1 |> as_gt() |> gt::gtsave("Resources/table1_demo.html")

```

Choosing and making figures

Match the figure to the variable: a histogram for one numeric variable, a bar chart for one categorical, a boxplot for numeric-by-category, a scatterplot for two numerics. Every plot gets a clear title, axis labels **with units**, `theme_minimal()`, and the teal accent. `ggsave()` writes it to disk at 300 dpi.

```
# Histogram of age
p_age <- ggplot(analysis_data, aes(x = age)) +
  geom_histogram(binwidth = 5, fill = course_teal, colour = "white") +
  labs(title = "Age distribution of study participants",
        x = "Age (years)", y = "Number of patients") +
  theme_minimal(base_size = 13)
ggsave("Resources/day3_hist_age.png", plot = p_age,
        width = 7, height = 5, dpi = 300)

# Bar chart of education (geom_bar counts categories for you)
p_edu <- ggplot(analysis_data, aes(x = education)) +
  geom_bar(fill = course_teal) +
  labs(title = "Educational attainment of participants",
        x = "Education level", y = "Number of patients") +
  theme_minimal(base_size = 13)

# Boxplot of SBP by treatment uptake
p_box <- ggplot(analysis_data, aes(x = treatment_uptake, y = sbp_mmhg)) +
  geom_boxplot(fill = course_teal, alpha = 0.6, na.rm = TRUE) +
  labs(title = "Systolic BP by treatment uptake",
        x = "On hypertension treatment?", y = "Systolic BP (mmHg)") +
  theme_minimal(base_size = 13)

# Scatterplot of SBP vs BMI with a linear trend
p_scatter <- ggplot(analysis_data, aes(x = bmi, y = sbp_mmhg)) +
  geom_point(alpha = 0.3, colour = course_teal) +
  geom_smooth(method = "lm", se = TRUE, colour = "grey20") +
  labs(title = "Systolic BP vs BMI",
        x = "BMI (kg/m^2)", y = "Systolic BP (mmHg)") +
  theme_minimal(base_size = 13)
```

A ggplot is built by adding layers with `+`: data and aesthetic mappings (`aes()`) first, then a geometry (`geom_*`), then labels and theme.

Clinical interpretation: an upward-sloping line in the scatterplot suggests higher BMI tends to accompany higher systolic BP - consistent with obesity as a risk factor. Remember: this is association, not proof of causation.

Exporting a summary table

Share numbers with co-authors who do not use R by writing a plain CSV.

```
write_csv(numeric_summary, "Resources/day3_numeric_summary.csv")
```

Key takeaways

- Always pass `na.rm = TRUE` to summary functions; a stray NA is the usual cause of an NA result.
- Prefer median (IQR) over mean (SD) for skewed clinical variables.
- Make missing values visible in tables with `useNA = "ifany"`.
- Pick the correct `prop.table()` margin for the question you are asking.
- `gtsummary` builds Table 1 in a few lines; `ggplot2` + `ggsave()` make and save publication figures at 300 dpi.

Day 3 exercise

Open Practicals/day3_exercise.R. Produce a grouped numeric summary by treatment_uptake, two correctly-labelled frequency tables, one cross-tab with the appropriate margin, a gtsummary Table 1, and at least two figures saved to Resources/.

Chapter 4 (Session 4): Common Medical Statistical Tests

- > **Note on sequencing.** The current slide deck keeps **Session 4 to statistical tests
- > only** (t-test, Wilcoxon, ANOVA, chi-square, Fisher, correlation) and introduces
- > **regression — linear then logistic — in Session 5.** This chapter still includes a
- > logistic-regression primer at the end; treat that primer as your bridge into Session 5.

Session 4 moves from describing the data to asking questions of it with statistical tests. We work in the analytic population - patients already diagnosed with hypertension.

```
library(tidyverse)
library(broom) # turns model output into a tidy data frame

analysis_data <- readRDS("Data/analysis_data.rds")

dx <- filter(analysis_data, htn_diagnosed == "Yes")
nrow(dx) # ~1089 patients in the analysis
table(dx$treatment_uptake) # how many took up treatment vs not
```

Common mistake: running the analysis on all 1,500 patients. That mixes in undiagnosed people, for whom the outcome is undefined, and biases every result.

p-values, in one paragraph

A **p-value** is the probability of seeing data at least as extreme as yours *if the null hypothesis were true*. A small p-value (conventionally below 0.05) means the data are surprising under the null, so we "reject" it. A p-value is **not** the probability the null is true, and it is **not** a measure of effect size. In large samples, even trivial differences become "significant" - so always pair the p-value with the size of the effect.

Checking normality

Many classic ("parametric") tests assume roughly normal data. Look first, then test.

```
hist(dx$age, breaks = 30, col = course_teal, border = "white",
     main = "Distribution of Age", xlab = "Age (years)")

qqnorm(dx$age, main = "Q-Q Plot: Age"); qqline(dx$age, col = course_teal, lwd = 2)

shapiro.test(dx$age) # H0: data ARE normal; small p -> reject normality
shapiro.test(dx$sbp_mmhg)
```

What the output means: points hugging the Q-Q line indicate approximate normality; points curving away at the ends indicate skew. **Caution:** with ~1,089 rows the Shapiro-Wilk test is so powerful it

flags trivial departures as "significant". Trust a near-straight Q-Q plot and a symmetric histogram more than the Shapiro p-value here. Rule of thumb: symmetric and large sample -> parametric is fine; clearly skewed or small sample -> use the non-parametric test.

Two groups, continuous outcome: t-test / Wilcoxon

```
# Parametric: Welch two-sample t-test (does not assume equal variances)
t.test(age ~ treatment_uptake, data = dx)

# Non-parametric equivalent (compares ranks; robust to skew/outliers)
wilcox.test(age ~ treatment_uptake, data = dx)
```

The formula `age ~ treatment_uptake` reads "age explained by treatment uptake".

What the output means: read three things from the t-test - the two group means, the 95% confidence interval for their difference, and the p-value. If both tests agree (they usually do for large, symmetric data), report the t-test with means; if they disagree, trust the non-parametric test and report medians.

More than two groups: ANOVA

Running many pairwise t-tests inflates the false-positive rate. ANOVA tests all groups at once with a single honest p-value.

```
aov_age <- aov(age ~ education, data = dx)
summary(aov_age)      # the F-statistic and its p-value
TukeyHSD(aov_age)     # which specific pairs differ (corrected for multiplicity)
```

What the output means: a small ANOVA p-value says *some* groups differ but not which; the Tukey post-hoc test identifies the specific pairs. The non-parametric analogue is `kruskal.test()`.

Two categorical variables: chi-square / Fisher

```
uptake_diabetes <- table(dx$treatment_uptake, dx$diabetes)
addmargins(uptake_diabetes)      # table with row/column totals

chisq.test(uptake_diabetes)      # H0: the variables are independent
chisq.test(uptake_diabetes)$expected # check expected counts are all >= 5
fisher.test(uptake_diabetes)     # use when expected counts are small
```

What the output means: a small p-value means uptake and diabetes are associated; the table shows the direction. The chi-square test is valid only when expected counts are large enough (a common rule: all expected cells at least 5); otherwise use Fisher's exact test. Neither test gives an effect size - the odds ratio does.

Correlation: two continuous variables

```
cor.test(dx$sbp_mmhg, dx$bmi, method = "pearson") # linear; assumes normality
cor.test(dx$sbp_mmhg, dx$bmi, method = "spearman") # rank-based; robust
```

What the output means: the coefficient r runs from -1 to +1: roughly 0.0-0.3 weak, 0.3-0.7 moderate, 0.7-1.0 strong; the sign gives direction. In big samples even a tiny r (say 0.08) can be "significant" yet clinically trivial - judge r , not just the p -value. Correlation is not causation.

Introduction to logistic regression

Our outcome is binary (Yes/No), so we use **logistic regression**, not linear regression. `family = binomial` tells `glm()` the outcome is 0/1. Because `treatment_uptake` has reference "No", the model estimates the odds of "Yes".

```
# Single categorical predictor
m_diab <- glm(treatment_uptake ~ diabetes, data = dx, family = binomial)
exp(coef(m_diab))      # odds ratios (the raw coefficients are log-odds)
exp(confint(m_diab))   # their 95% confidence intervals
```

Interpreting an odds ratio (OR): $OR = 1$ means no effect; $OR > 1$ means higher odds of uptake; $OR < 1$ means lower odds. A 95% CI that **excludes 1** is statistically significant. Phrasing: "Patients with diabetes had about X times the odds of treatment uptake compared with those without (OR X.X, 95% CI a-b)."

For a continuous predictor, the OR is "per one-unit increase". One year of age is a small step, so rescale to something meaningful:

```
m_age <- glm(treatment_uptake ~ age, data = dx, family = binomial)
exp(coef(m_age))
exp(coef(m_age)["age"] * 10)  # OR per 10-year increase, easier to communicate
```

From univariable to adjusted models

Patients differ in many ways at once. A **multivariable** model estimates the effect of each predictor while holding the others constant ("adjusting" for them).

```
m_multi <- glm(treatment_uptake ~ age + sex + diabetes + residence,
               data = dx, family = binomial)
exp(cbind(OR = coef(m_multi), confint(m_multi))) # adjusted ORs with 95% CI
nobs(m_multi) # rows actually used: glm() drops rows missing ANY model variable
```

`broom::tidy()` produces a clean, report-ready table in one call:

```
tidy(m_multi, exponentiate = TRUE, conf.int = TRUE)
```

What the output means: these are **adjusted** ORs - comparisons between patients who are otherwise similar on the other variables. An OR can shrink, grow, or even flip after adjustment; that change is the signature of confounding. Note also that `glm()` uses complete cases by default, so check how many rows survived with `nobs()`.

Matching the question to the test

Question	Test
Two groups, continuous	<code>t.test()</code> (or <code>wilcox.test()</code> if skewed)
More than two groups, continuous	<code>aov()</code> (or <code>kruskal.test()</code> if skewed)

Question	Test
Two categorical variables	<code>chisq.test()</code> (or <code>fisher.test()</code> if sparse)
Two continuous variables	<code>cor.test()</code> (Pearson or Spearman)
Binary outcome and its drivers	<code>glm(..., family = binomial)</code> -> odds ratios

Key takeaways

- Define the analytic population explicitly (`htn_diagnosed == "Yes"`) and analyse it.
- A p-value measures surprise under the null, not effect size or truth.
- Look at the distribution before choosing a parametric vs non-parametric test.
- Choose the test that matches your question and variable types.
- Logistic regression gives odds ratios; a 95% CI excluding 1 is significant.
- Adjusted ORs hold other variables constant; `broom::tidy()` formats them cleanly.

Day 4 exercise

Open `Practicals/day4_exercise.R`. On the diagnosed subset, run one appropriate two-group test, one chi-square (or Fisher) test, one correlation, and at least two univariable logistic models. Then fit a small adjusted model and produce a tidy OR table with confidence intervals.

Chapter 5 (Session 5): Introduction to Regression & Interpreting Output (Capstone)

Session 5 introduces regression and brings everything together: linear regression for a continuous outcome, logistic regression for a Yes/No outcome, confounding and adjustment, the final adjusted ORs, and writing them up.

```
library(dplyr); library(ggplot2); library(broom)
library(gtsummary); library(car); library(pROC)

set.seed(2025) # makes any randomness reproducible

analysis_data <- readRDS("Data/analysis_data.rds")
dx <- filter(analysis_data, htn_diagnosed == "Yes")
nrow(dx) # ~1089 - report this n
levels(dx$treatment_uptake) # c("No", "Yes") -> "No" is the reference
```

Model-building philosophy

Build the model from **clinical knowledge, not from p-values**. Decide your predictors before looking at the data, based on biological plausibility, known confounders from the literature (age, sex, education, residence), and the study's stated determinants (access via distance, awareness via knowledge). Then fit one pre-specified model. This avoids "data dredging" - the practice of running everything and reporting only what came out significant, which inflates false positives and rarely replicates.

```
# A crude (unadjusted) model, used to demonstrate confounding
crude_residence <- glm(treatment_uptake ~ residence,
                      data = dx, family = binomial(link = "logit"))

# The full, pre-specified multivariable model (our main analysis)
model_full <- glm(
  treatment_uptake ~ age + sex + education + residence + diabetes +
    family_history_htn + health_insurance + knowledge_score +
    distance_to_facility_km,
  data = dx, family = binomial(link = "logit")
)
summary(model_full) # coefficients are on the log-odds scale
```

Confounding: crude vs adjusted

A confounder distorts a crude association. Compare the crude and adjusted OR for residence.

```
crude_or <- exp(coef(crude_residence))["residenceUrban"]
adj_or <- exp(coef(model_full))["residenceUrban"]
cat("Crude OR (Urban vs Rural): ", round(crude_or, 2), "\n")
cat("Adjusted OR (Urban vs Rural):", round(adj_or, 2), "\n")
```

Clinical interpretation: if the OR moves noticeably (a rule of thumb is more than a 10% change) when the other variables are added, those variables were confounding the crude residence effect. The **adjusted** OR is the one you report.

Interaction (effect modification)

Interaction asks "does the effect of A differ across levels of B?" - for example, does the effect of diabetes change with age? Add the product term and compare nested models with a likelihood ratio test.

```
model_interax <- glm(
  treatment_uptake ~ age + sex + education + residence + diabetes +
    family_history_htn + health_insurance + knowledge_score +
    distance_to_facility_km + diabetes:age,
  data = dx, family = binomial(link = "logit")
)
anova(model_full, model_interax, test = "LRT")
```

Clinical interpretation: here the interaction is expected to be non-significant ($p > 0.05$). When that happens, keep the simpler `model_full` - it is easier to interpret. Do not chase interactions, and never interpret a main effect in isolation while its interaction term is still in the model.

Variable selection, with caution

```
model_step <- step(model_full, direction = "both", trace = 0) # lowest-AIC search
AIC(model_full, model_step)
model_final <- model_full # our decision: keep the pre-specified model
```

Caution: stepwise selection produces optimistic p-values and CIs (not corrected for the search), gives different "selected" models on different datasets, and can drop a known confounder just because $p > 0.05$ - which reintroduces bias. For an explanatory study about determinants, keep clinically chosen variables even if non-significant. Reserve `step()`/AIC mostly for prediction tasks.

Diagnostics

```
# Multicollinearity: VIF < 5 fine, 5-10 worth a look, > 10 serious.
car::vif(model_final)

# Influential points via Cook's distance
aug <- broom::augment(model_final)
infl_cut <- 4 / nrow(aug)
sum(aug$.cooks > infl_cut, na.rm = TRUE) # how many flagged

# Discrimination: AUC (0.5 = chance, 0.7-0.8 = acceptable, > 0.8 = good)
roc_obj <- pROC::roc(response = model_final$y,
                     predictor = fitted(model_final), quiet = TRUE)
cat("Model AUC:", round(as.numeric(pROC::auc(roc_obj)), 3), "\n")
```

What the output means: VIF values near 1-2 mean predictors are not redundant. Flagged influential points should be inspected, not deleted - check whether they are data-entry errors or genuine extreme patients, and run a sensitivity analysis if results change. The **AUC of about 0.71** indicates acceptable discrimination: the model separates those who do and do not take up treatment better than chance.

The final model: adjusted odds ratios

```
or_table <- broom::tidy(model_final, exponentiate = TRUE, conf.int = TRUE)
print(or_table, n = Inf)
```

On this dataset the final adjusted ORs (n = 992 complete cases; overall uptake about 47%) are:

Predictor	aOR	95% CI	Direction
Age (per year)	1.03	1.02-1.04	higher uptake
Sex: Male (vs Female)	0.74	0.56-0.97	lower uptake
Education (linear trend)	1.96	1.39-2.77	higher uptake with more education
Residence: Urban (vs Rural)	1.87	1.41-2.49	higher uptake
Diabetes: Yes	3.56	1.46-9.61	higher uptake
Family history of HTN: Yes	1.91	1.44-2.54	higher uptake
Health insurance: Yes	2.05	1.54-2.74	higher uptake
Knowledge score (per point)	1.10	1.06-1.14	higher uptake
Distance to facility (per km)	0.98	0.96-1.01	not significant

Reading the table: diabetes shows the largest effect (over three times the odds) but the widest CI, because diabetics are a smaller subgroup - a reminder that a big point estimate with a wide interval is less precise. Distance is in the expected (protective-against-uptake) direction but is not significant after adjustment. Model discrimination is acceptable (AUC ~ 0.71).

A publication-ready table and forest plot follow the same model:

```
tbl <- gtsummary::tbl_regression(model_final, exponentiate = TRUE) |>
  gtsummary::bold_p() |>
  gtsummary::modify_caption("***Adjusted odds ratios for treatment uptake**")

plot_df <- or_table |> filter(term != "(Intercept)")
forest <- ggplot(plot_df, aes(x = estimate, y = reorder(term, estimate))) +
  geom_vline(xintercept = 1, linetype = "dashed", colour = "grey50") +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.2,
    colour = "#0D7377") +
  geom_point(size = 2.6, colour = "#0D7377") +
  scale_x_log10() +
  labs(title = "Adjusted odds ratios for treatment uptake",
    x = "Adjusted odds ratio (log scale)", y = NULL) +
  theme_minimal(base_size = 12)
ggsave("Resources/forest_plot_or.png", plot = forest, width = 8, height = 5, dpi = 300)
```

Writing a Results section

Report the analytic n, the outcome frequency, the adjusted ORs with CIs, the directions, and the model's discrimination. State your significance rule once. Here is a worked example using the real numbers:

> Of the 1,089 adults with diagnosed hypertension, 992 had complete data for the multivariable model, and approximately 47% reported uptake of antihypertensive treatment. In the adjusted logistic regression model, diabetes was associated with markedly higher odds of uptake (aOR 3.56, 95% CI 1.46-9.61), as were having health insurance (aOR 2.05, 95% CI 1.54-2.74), a positive family history of hypertension (aOR 1.91, 95% CI 1.44-2.54), and urban (vs rural) residence (aOR 1.87, 95% CI 1.41-2.49). Each additional point of hypertension knowledge (aOR 1.10, 95% CI 1.06-1.14) and each year of age (aOR 1.03, 95% CI 1.02-1.04) modestly increased the odds of uptake, and higher educational attainment showed a positive trend (aOR per level 1.96, 95% CI 1.39-2.77). Male sex was associated with lower odds of uptake than female sex (aOR 0.74, 95% CI 0.56-0.97). Greater distance to the facility was in the expected protective-against-uptake direction but was not statistically significant after adjustment (aOR 0.98, 95% CI 0.96-1.01). The model showed acceptable discrimination (area under the ROC curve 0.71). Associations whose 95% confidence interval excluded 1.00 were considered statistically significant.

Reproducibility: the capstone message

- Use `set.seed()` whenever randomness is involved.
- Use **relative paths** (or `here::here(...)`); never hard-code `C:/Users/yourname/...`
- Keep a clear folder structure: `Data/`, `Scripts/`, `Resources/`, `References/`.
- Save every table and figure to disk so the report can be rebuilt from code.
- Record your environment with `sessionInfo()` for your supplementary materials.
- For a one-click report, move the analysis into R Markdown (`.Rmd`) or Quarto (`.qmd`): knitting re-runs the code and produces a Word/PDF/HTML report with text, tables, and figures together - the gold standard for reproducible clinical reporting.

```
sessionInfo()
# writeLines(capture.output(sessionInfo()), "References/session_info.txt")
```

Key takeaways

- Pre-specify the model from clinical knowledge; fit it once.
- Compare crude and adjusted ORs to detect confounding.
- Test interactions with a likelihood-ratio test; keep the simpler model unless it is both supported and meaningful.
- Treat automated variable selection with caution in explanatory studies.
- Check VIF, influential points, and AUC; interpret, do not just delete.
- Report adjusted ORs with CIs and directions, and make the whole analysis reproducible.

Day 5 exercise

Open Practicals/day5_exercise.R. Fit the pre-specified multivariable model on the diagnosed subset, demonstrate confounding for one variable, test one interaction, check VIF and AUC, produce a tidy OR table and a forest plot, and write a short Results paragraph in your own words using your computed numbers.

Common Errors and How to Fix Them

Symptom	Likely cause	Fix
could not find function "read_csv"	Package not loaded	Run <code>library(tidyverse)</code> (or <code>library(readxl)</code> for Excel)
A mean/sd/median returns NA	Missing values in the column	Add <code>na.rm = TRUE</code>
object 'sbp' not found	Typo or wrong case; object not created yet	R is case-sensitive; check spelling and run the line that creates it
Error: '...' does not exist in current working directory	Wrong path / wrong working directory	Use an RStudio Project and relative paths; check <code>getwd()</code>
Import shows more rows than expected (e.g. 1503)	Duplicate records	<code>raw <- distinct(raw)</code>
A frequency table seems to "lose" patients	<code>table()</code> drops NA silently	Use <code>table(x, useNA = "ifany")</code>
Odds ratio looks backwards	Wrong factor reference level	Set levels so the reference is first, e.g. <code>factor(x, levels = c("No", "Yes"))</code>
Model uses fewer rows than your dataset	<code>glm()</code> drops rows missing any model variable	Check <code>nobs(model)</code> ; inspect missingness
non-numeric argument to ... on a number column	Column imported as character (stray text/whitespace)	Trim and convert: <code>str_trim()</code> then <code>as.numeric()</code>
<code>install.packages()</code> runs every session	Confusing install with load	Install once ; <code>library()</code> each session
= used for assignment, confusing later	= vs <-	Use <- for assignment (Alt+-)
Date column will not sort or compute	Stored as text	Parse with <code>lubridate::parse_date_time()</code> then <code>as_date()</code>

Glossary

- **Object:** a named container for a value or dataset, created with `<-` (e.g. `age <- 60`).
- **Vector:** an ordered set of values of the same type, built with `c()`.
- **Data frame (tibble):** a table where rows are observations (patients) and columns are variables.
- **Function:** a named operation that takes inputs (arguments) and returns a result, e.g. `mean(x, na.rm = TRUE)`.
- **Argument:** a value you pass to a function, like `na.rm = TRUE`.
- **Factor:** a categorical variable with a fixed set of **levels**; the first level is the **reference**.
- **Reference level:** the category other categories are compared against in a model.
- **Package:** a shareable bundle of functions; installed once, loaded each session with `library()`.
- **tidyverse:** a coherent family of packages for importing, wrangling, and plotting data (includes `dplyr`, `ggplot2`, `readr`).
- **Pipe (`|>`):** passes the result on its left into the function on its right; read as "and then".
- **NA:** R's marker for a missing value.
- **na.rm:** an argument that tells a summary function to ignore missing values.
- **Working directory:** the folder R is currently looking at for files.
- **Descriptive statistics:** numbers that summarise a sample (mean, median, SD, IQR, counts, percentages).
- **Hypothesis test:** a procedure that weighs the data against a null hypothesis (t-test, chi-square, etc.).
- **p-value:** the probability of data as extreme as yours if the null hypothesis were true; small means surprising under the null.
- **Logistic regression:** a model for a binary (Yes/No) outcome, fitted with `glm(..., family = binomial)`.
- **Odds ratio (OR):** the multiplicative change in the odds of the outcome; $OR > 1$ increases the odds, $OR < 1$ decreases it, $OR = 1$ means no effect.
- **Confidence interval (95% CI):** a range that, with 95% confidence, contains the true value; a CI excluding 1 (for an OR) signals significance.
- **Confounding:** a third variable distorting the association between an exposure and an outcome; addressed by adjustment.
- **Interaction (effect modification):** when the effect of one variable depends on the level of another.
- **AUC:** area under the ROC curve; a measure of how well a model discriminates (0.5 = chance, ~0.7-0.8 = acceptable).
- **VIF:** variance inflation factor; flags predictors that are too correlated with each other (collinearity).
- **Reproducibility:** the property that anyone can re-run your code and obtain the same results; supported by scripts, relative paths, saved outputs, and `sessionInfo()`.

Final Assignment and Encouragement

The final assignment

Your capstone is a complete, reproducible mini-analysis on the hypertension dataset:

1. **Clean** the raw data to your own `analysis_data.rds` (Day 2 skills).
2. **Describe** the diagnosed-hypertensive sample with a `gtsummary` Table 1 and at least two figures (Day 3).
3. **Test** at least two pre-specified hypotheses with the correct tests (Day 4).
4. **Model** treatment uptake with a pre-specified multivariable logistic regression, report adjusted ORs with 95% CIs and the AUC, and produce a forest plot (Day 5).
5. **Write** a short Results paragraph in clinical-manuscript style, using your own computed numbers.
6. **Make it reproducible:** relative paths, saved outputs, and `sessionInfo()` recorded.

You will be marked on correct method and sound interpretation, not on matching a number to the second decimal place. Exact decimals shift trivially with R and package versions; confidence intervals and directions are what matter.

Keep practising

You have just done what many clinicians never get to do: turn raw, imperfect data into a defensible answer to a real question - and done it transparently, so others can check your work. That is the whole point of `#ClearDataClearImpact`.

The skills compound. Re-run the demo scripts. Try the analysis on a variable we did not cover. Break things on purpose and read the error messages; they are teachers, not enemies. Keep the cheat sheet (`R_command_reference_sheet.md`) and this handbook beside you, and reach out when you are stuck.

Clear data, clear impact. Well done, and keep going.